Computer Graphics — Question Bank Solutions

Unit 1 – Unit 3 (Mid-Semester Examination)

ITM SLS Baroda University — Course Code: C2510C4 — B.Tech Semester 5

Section A — Multiple Choice Questions ($20 \times 1 = 20$ Marks)

Q	Ans	Reason
1	b	Real-time graphics are redrawn so quickly (typically 30–60 frames per second or more) that motion looks smooth and continuous, e.g. games and simulations. Option (a) describes offline/pre-rendered graphics.
2	b	Raster images store a *grid of pixels*; vector images store *mathematical equations* (points, lines, curves, polygons).
3	b	*Rasterization* converts triangles (geometry) into fragments/pixels on the screen grid. The fragment shader only colours them; the frame buffer only stores them.
4	c	In OLED every pixel is self-emissive. Switching a pixel off gives true black. LCD uses a backlight, so black is never fully dark.
5	b	CMYK is subtractive (inks absorb light) and is the standard model for printing. RGB is additive; HSV is a perceptual model.
6	c	Hue is the actual colour type, written as an angle (0°–360°) on the colour wheel. Saturation is vividness; Value is brightness.
7	b	Human vision does not respond linearly to light intensity (it is more sensitive to dark tones), so images are gamma-encoded to match perception and use bits efficiently.
8	b	Bresenham's algorithm uses only integer addition and subtraction. DDA needs floating-point division and rounding.
9	b	steps $= \max(\lvert dx\rvert, \lvert dy\rvert)$ so that the faster-changing axis moves exactly one pixel per step and no gaps appear.
10	c	A circle has 8-way symmetry (about both axes and the lines $y = \pm x$), so only one octant must be computed.
11	b	The slope of the ellipse boundary changes from $\lvert m\rvert < 1$ to $\lvert m\rvert > 1$, so the unit-step axis must switch from x to y (two regions).
12	b	SSAA (super-sampling) renders at a much higher resolution and then down-samples (averages) to the screen resolution.
13	b	Even–Odd rule: an *odd* number of boundary crossings of a ray from the point means the point is inside.
14	b	Flood fill replaces all connected pixels that have the seed pixel's original colour; boundary fill stops at a given boundary colour.
15	c	In homogeneous coordinates a 2D point (x, y) is written $(x, y, 1)$ (generally (wx, wy, w) with $w = 1$).
16	b	Matrix multiplication is not commutative ($AB \neq BA$), so the order of composite transformations matters.
17	b	To rotate about a pivot: translate the pivot to the origin $\rightarrow$ rotate $\rightarrow$ translate back.

Q	Ans	Reason
18	b	Both end-point codes 0000 means both points are inside the window: trivial accept.
19	b	Basic Sutherland–Hodgman clips against one boundary of a *convex* clip window at a time (concave windows give wrong results).
20	b	Affine transformations preserve parallelism of lines (and straightness) but not necessarily lengths or angles.

Section B — Short Answer Questions (3 Marks Each)

Unit 1

B1. Differentiate between raster graphics and vector graphics, with one example of each. [3 Marks]

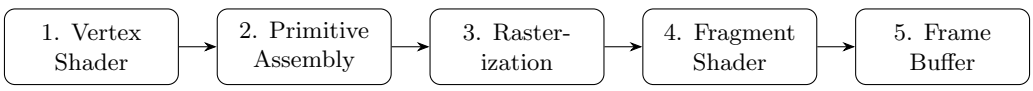
Answer:

Raster graphics (bitmap images) store an image as a rectangular grid of pixels, where every pixel holds its own colour value. **Vector graphics** store an image as a set of geometric objects (points, lines, curves, polygons) described by mathematical equations together with attributes such as stroke colour, fill colour and thickness.

Basis	Raster Graphics	Vector Graphics
Representation	Grid of pixels with a colour value for each pixel	Mathematical equations/primitives (lines, curves, polygons)
Resolution	Resolution dependent (fixed number of pixels)	Resolution independent
Scaling	Becomes blurred/pixelated when enlarged	Can be scaled to any size without loss of quality
File size	Grows with image dimensions and colour depth	Small; depends on the number of objects
Realism	Excellent for photographs and complex shading	Poor for photo-realism; best for flat/clean shapes
Editing	Edited pixel by pixel	Each object can be edited independently
Common formats	JPEG, PNG, BMP, GIF	SVG, AI, EPS, PDF
Example	A digital photograph (JPEG), edited in Photoshop	A company logo (SVG), drawn in Illustrator

B2. List the five stages of the GPU rendering pipeline, in the correct order. [3 Marks]

Answer:



1. **Vertex Shader** — processes every vertex; applies model, view and projection transformations to give clip-space positions (programmable).
2. **Primitive Assembly** — joins vertices into triangles/lines/points, then clips them to the view volume and discards back faces.
3. **Rasterization** — converts each triangle into *fragments* (candidate pixels) and interpolates attributes such as colour, depth and texture coordinates.
4. **Fragment Shader** — computes the final colour of each fragment using texture lookup, lighting and effects (programmable).

5. **Frame Buffer (Output Merger)** — performs depth and stencil tests and blending; the surviving colour is written to the frame buffer and displayed.

B3. Why does CMYK use a separate black (K) ink channel instead of relying only on mixing Cyan, Magenta and Yellow? [3 Marks]

Answer:

In theory, equal amounts of C, M and Y absorb all of red, green and blue light and produce black. In practice a separate **K (key/black)** ink is used because:

1. **Impure pigments:** real inks are not perfect absorbers, so 100% C + M + Y gives a muddy dark brown or grey, not a true black.
2. **Ink saving and drying time:** reproducing a dark tone with three inks uses up to 300% ink coverage, which soaks the paper, dries slowly and smudges. One black ink is cheaper and uses far less ink.
3. **Sharp text and fine lines:** black text printed with three inks suffers from mis-registration (colour fringes). A single black plate keeps the edges crisp.
4. **Better contrast and shadow detail:** K gives a deeper, neutral black and improves the density of dark regions.

The black component is extracted as $K = \min(C, M, Y)$ (under-colour removal), and the remaining colour is reduced accordingly: $C' = C - K$, $M' = M - K$, $Y' = Y - K$.

Unit 2

B4. Write the formula used in the DDA algorithm to calculate the number of steps for a line, and explain each term. [3 Marks]

Answer:

For a line from (x_1, y_1) to (x_2, y_2):

$$\boxed{\text{steps} = \max\big(|dx|,\, |dy|\big)}, \qquad dx = x_2 - x_1, \quad dy = y_2 - y_1.$$

- dx and dy — the horizontal and vertical change between the two end points.
- $|dx|,\ |dy|$ — absolute values, so that the formula works for any direction of the line.
- $\max(\cdot, \cdot)$ — the larger of the two changes is chosen, so that the axis with the larger change advances by exactly one pixel per step. This guarantees that no pixel gaps are left in the line.
- steps — the total number of increments (the number of points generated after the first point).

The increments per step are then $x_{inc} = dx/\text{steps}$ and $y_{inc} = dy/\text{steps}$, and each step does $x \leftarrow x + x_{inc}$, $y \leftarrow y + y_{inc}$ and plots $(\text{round}(x), \text{round}(y))$.

Example: for $(2, 2) \rightarrow (10, 6)$: $dx = 8$, $dy = 4$, so steps $= \max(8, 4) = 8$, $x_{inc} = 1$, $y_{inc} = 0.5$.

B5. Differentiate between the boundary-fill algorithm and the flood-fill algorithm. [3 Marks]

Answer:

Both algorithms start from a **seed pixel** inside a region and spread to neighbouring pixels (4- or 8-connected), usually using recursion or an explicit stack.

Basis	Boundary Fill	Flood Fill
Stopping condition	Stops when a pixel of the specified *boundary colour* is reached	Stops when a pixel's colour is different from the *old (seed) colour*
Pixels replaced	Pixels that are neither the boundary colour nor the fill colour	All connected pixels having the original interior colour
Region requirement	Region must be bounded by a single, known boundary colour	Interior must be of one colour; the border may be of many colours
Typical test	`if (getpixel != boundary && getpixel != fill)`	`if (getpixel == oldcolor)`
Use	Filling closed shapes with a uniform outline	Recolouring an area (paint-bucket tool)

Both can be implemented with 4-connectivity (up, down, left, right) or 8-connectivity (adding the diagonals); 8-connected fill can leak through diagonal gaps in a boundary.

B6. What is anti-aliasing? Name any two anti-aliasing techniques used in computer graphics. [3 Marks]

Answer:

Aliasing is the jagged, "staircase" appearance of slanted lines and curved edges, caused by representing a continuous shape on a discrete pixel grid (under-sampling). **Anti-aliasing** is the set of techniques used to reduce or hide this effect by smoothing the edge pixels, i.e. blending the edge colour with the background in proportion to the area of the pixel covered by the shape.



Aliased (jagged) Anti-aliased (smooth; illustrative)

Two common techniques:

1. **SSAA (Super-Sampling Anti-Aliasing):** the scene is rendered at a much higher resolution and then down-sampled (averaged) to the display resolution. Best quality but very costly.
2. **MSAA (Multi-Sampling Anti-Aliasing):** several coverage samples per pixel are taken, but shading is done only once per pixel and only along polygon edges, so it is cheaper than SSAA.

(Other methods: Wu's line algorithm, FXAA, TAA.)

Unit 3

B7. Write the homogeneous-coordinate transformation matrix for a 2D translation by (t_x, t_y). [3 Marks]

Answer:

A point (x, y) is written as the column vector $(x, y, 1)^T$ in homogeneous coordinates. Translation by

(t_x, t_y) gives $x' = x + t_x$, $y' = y + t_y$, which in matrix form is

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}}_{T(t_x,t_y)} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}.$$

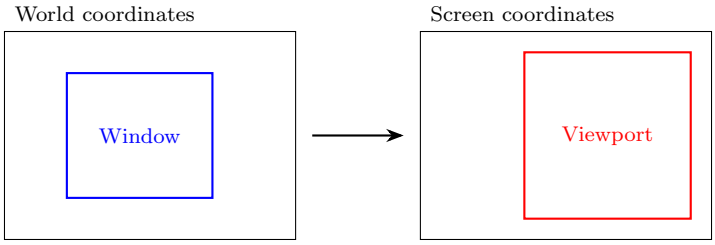
- **Inverse:** $T^{-1}(t_x, t_y) = T(-t_x, -t_y)$.
- **Composition:** $T(t_{x1}, t_{y1})\, T(t_{x2}, t_{y2}) = T(t_{x1} + t_{x2},\, t_{y1} + t_{y2})$.
- **Why homogeneous coordinates?** In ordinary 2×2 form translation is an *addition*, while rotation and scaling are *multiplications*. With the third coordinate all of them become 3×3 matrix products, so any sequence of transformations can be combined into a single matrix.

Example: translating $(3, 4)$ by $(2, -1)$ gives $(5, 3)$.

B8. Define "window" and "viewport". Why is the window-to-viewport transformation needed? [3 Marks]

Answer:

- **Window:** a rectangular area in the *world coordinate system* that selects which part of the scene is to be displayed.
- **Viewport:** a rectangular area in the *device (screen) coordinate system* in which the contents of the window are displayed.



Need: World coordinates and device coordinates have different units, origins and ranges. The window-to-viewport transformation maps every point (x_w, y_w) of the window to the corresponding point (x_v, y_v) of the viewport so that relative positions are preserved:

$$x_v = x_{vmin} + (x_w - x_{wmin})\, s_x, \qquad y_v = y_{vmin} + (y_w - y_{wmin})\, s_y,$$

$$s_x = \frac{x_{vmax} - x_{vmin}}{x_{wmax} - x_{wmin}}, \qquad s_y = \frac{y_{vmax} - y_{vmin}}{y_{wmax} - y_{wmin}}.$$

It makes zooming and panning possible (change the window), lets the same scene be shown on any screen size or in several viewports, and keeps the picture undistorted if $s_x = s_y$.

B9. State the four cases considered while clipping a single polygon edge in the Sutherland–Hodgman algorithm. [3 Marks]

Answer:

The polygon is clipped against one clip boundary at a time. Each polygon edge goes from the start vertex S to the end vertex P and is classified by the position of S and P relative to that boundary; I denotes the intersection of the edge with the boundary.

Case	S	P	**Output to the new vertex list**
1	inside	inside	P only
2	inside	outside	the intersection point I only (edge is leaving)
3	outside	outside	nothing
4	outside	inside	the intersection point I, then P (edge is entering)

The output list of one boundary becomes the input list for the next boundary (left, right, bottom, top), and the list left after the last boundary is the clipped polygon.

Section C — Medium Answer Questions (5 Marks Each)

Unit 1

C1. Explain the RGB, CMY and HSV colour models, along with their typical use cases. [5 Marks]

Answer:

A **colour model** is a mathematical system that describes every colour as a tuple of numbers.

1. RGB model (Red, Green, Blue) — additive

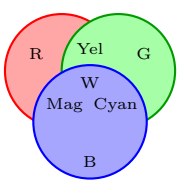
- Colours are created by *adding* light of the three primaries: Red, Green and Blue. Each component lies in $[0, 1]$ (or 0–255).
- $(0, 0, 0)$ = black, $(1, 1, 1)$ = white. Mixing: R + G = Yellow, G + B = Cyan, R + B = Magenta, R + G + B = White.
- The model is a unit cube; the diagonal from black to white is the grey axis. A 24-bit pixel has 8 bits per channel, giving $2^{24} \approx 16.7$ million colours.
- **Use cases:** monitors, TVs, mobile screens, digital cameras, scanners, web and game graphics.

2. CMY model (Cyan, Magenta, Yellow) — subtractive

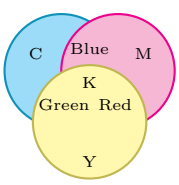
- Inks or dyes *absorb* (subtract) light from white paper. Cyan absorbs red, Magenta absorbs green, Yellow absorbs blue.
- Conversion from RGB: $C = 1 - R,\ M = 1 - G,\ Y = 1 - B$. Example: $(R, G, B) = (1, 0, 0)$ red $\Rightarrow (C, M, Y) = (0, 1, 1)$.
- A black ink (K) is added in practice, giving CMYK, with $K = \min(C, M, Y)$.
- **Use cases:** colour printing (inkjet, laser, offset), publishing, packaging.

3. HSV model (Hue, Saturation, Value) — perceptual

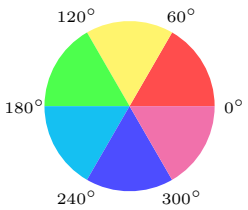
- **Hue (H):** the colour type, an angle 0°–360° on the colour wheel (red 0°, yellow 60°, green 120°, cyan 180°, blue 240°, magenta 300°).
- **Saturation (S):** purity of the colour, 0 (grey) to 1 (fully vivid).
- **Value (V):** brightness, 0 (black) to 1 (brightest).
- It matches the way humans describe colour, so a colour can be made lighter or less vivid by changing only V or S.
- **Use cases:** colour pickers, image-editing tools, colour-based segmentation/object tracking in computer vision.



RGB (additive)



CMY (subtractive)

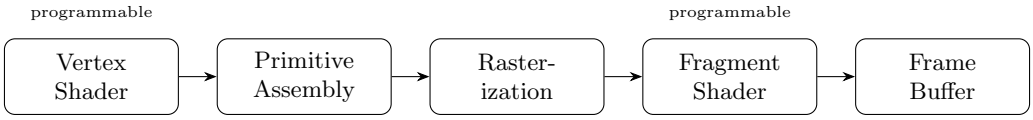


HSV hue wheel

C2. Describe the five stages of the GPU rendering pipeline, explaining briefly what happens at each stage. [5 Marks]

Answer:

The GPU rendering pipeline converts the 3D description of a scene (vertices, textures, lights) into the 2D pixels shown on the screen. Thousands of vertices and fragments are processed in parallel.



1. **Vertex Shader.** Input: vertices with attributes (position, normal, colour, texture coordinates). The shader multiplies each position by the *Model, View and Projection* matrices ($v' = P \cdot V \cdot M \cdot v$) to obtain clip-space coordinates. It may also compute per-vertex lighting or animation (skinning). Output: transformed vertices.
2. **Primitive Assembly.** Vertices are grouped into primitives (points, lines, triangles) according to the draw mode. The primitives are *clipped* against the view volume, the *perspective divide* (x/w, y/w, z/w) is applied, the *viewport transform* gives screen positions, and back-facing triangles can be removed (*culling*).
3. **Rasterization.** Decides which screen pixels each triangle covers and generates a *fragment* for each of them. Vertex attributes (colour, depth, texture coordinates) are interpolated across the triangle using barycentric coordinates.
4. **Fragment (Pixel) Shader.** Runs once for each fragment: samples textures, performs per-pixel lighting, shadows, fog and other effects, and outputs the final colour of the fragment.
5. **Frame Buffer / Output Merger.** Each fragment passes the depth test (Z-buffer, hidden surface removal) and stencil test, and is blended with the existing colour if it is transparent. The surviving colour is written to the frame buffer, which is sent to the display (double buffering avoids flicker).

Summary: stages 1 and 4 are programmable (shaders); stages 2, 3 and 5 are fixed-function hardware stages.

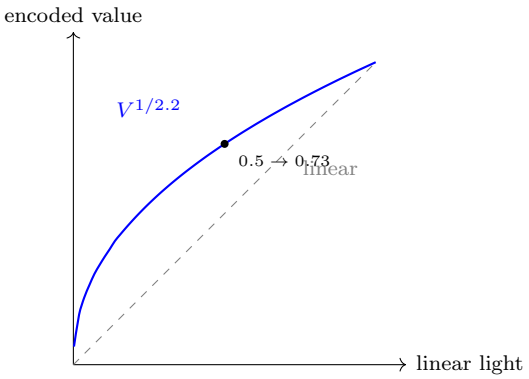
C3. What is gamma correction? Explain the difference between linear colour space and sRGB (gamma-encoded) colour space, and why this distinction matters. [5 Marks]

Answer:

Gamma correction is a non-linear transformation applied to pixel intensities so that the stored values, the display response and human brightness perception match each other.

$$V_{encoded} = V_{linear}^{1/\gamma} \text{ (gamma encoding)}, \qquad V_{linear} = V_{encoded}^{\gamma} \text{ (gamma decoding)}, \qquad \gamma \approx 2.2.$$

Reason: the human eye is far more sensitive to differences in dark tones than in bright tones; perceived brightness is roughly a power function of the light intensity. If intensities were stored linearly in 8 bits, too many codes would be wasted on bright tones and dark tones would show visible banding. Gamma encoding allocates more codes to the dark range. (Old CRT monitors also had a natural response $\approx V^{2.2}$ which encoding cancels.)



	Linear colour space	**sRGB (gamma-encoded) space**
Meaning of value	Proportional to physical light energy (0.5 = half the light)	Value encoded with $\approx \gamma = 2.2$ (0.5 looks like mid grey)
Behaviour	Physically correct for light mixing	Perceptually uniform: equal steps look equally different
Used for	Lighting, shading, blending, texture filtering, ray tracing	Storing images, textures, sending to display, web
Weakness	Needs more bits (floating point); looks too dark if shown directly	Maths on encoded values gives wrong results

Why the distinction matters:

- Lighting and blending must be computed in *linear* space; doing so in sRGB space gives wrong results. Example: the average of black and white is 0.5 in linear light, but the average of the encoded values 0 and 1 is 0.5 which, when displayed, corresponds to only $0.5^{2.2} \approx 0.22$ of the light, so the result looks too dark.
- The correct workflow is: decode sRGB textures to linear → do lighting/blending → encode to sRGB for display. Skipping a step gives harsh lighting, dark halos around transparent edges and poor anti-aliasing.

Unit 2

C4. Explain the DDA line-drawing algorithm. Using the algorithm, compute the pixel positions for a line from $(2, 2)$ to $(10, 6)$. [5 Marks]

Answer:

DDA (Digital Differential Analyzer) is an incremental scan-conversion algorithm based on the line equation $y = mx + c$ with slope $m = dy/dx$. It samples the line at unit intervals along the axis in which the line changes more, and computes the other coordinate by adding a constant increment:

$$y_{k+1} = y_k + m \quad (|m| \le 1,\ \Delta x = 1), \qquad x_{k+1} = x_k + \frac{1}{m} \quad (|m| > 1,\ \Delta y = 1).$$

Algorithm

1. Input the end points (x_1, y_1) and (x_2, y_2).
2. Compute $dx = x_2 - x_1$ and $dy = y_2 - y_1$.
3. steps $= \max(|dx|, |dy|)$.

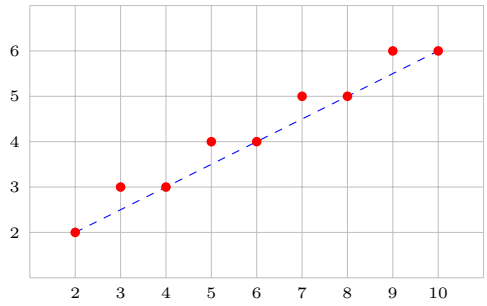
4. $x_{inc} = dx/\text{steps}$, $y_{inc} = dy/\text{steps}$.
5. Set $x = x_1$, $y = y_1$ and plot $(\text{round}(x), \text{round}(y))$.
6. Repeat `steps` times: $x \leftarrow x + x_{inc}$, $y \leftarrow y + y_{inc}$; plot $(\text{round}(x), \text{round}(y))$.

Numerical

$dx = 10 - 2 = 8$, $dy = 6 - 2 = 4$, steps $= \max(8, 4) = 8$, $x_{inc} = 8/8 = 1$, $y_{inc} = 4/8 = 0.5$.

Step k	x	y	Plotted pixel (rounded)
0	2	2.0	(2, 2)
1	3	2.5	(3, 3)
2	4	3.0	(4, 3)
3	5	3.5	(5, 4)
4	6	4.0	(6, 4)
5	7	4.5	(7, 5)
6	8	5.0	(8, 5)
7	9	5.5	(9, 6)
8	10	6.0	(10, 6)

(a value .5 is rounded up). **Pixels:** $(2,2), (3,3), (4,3), (5,4), (6,4), (7,5), (8,5), (9,6), (10,6)$.



Advantages and disadvantages

- Simple, and faster than directly solving $y = mx + c$ at each x (no multiplication per pixel).
- Uses floating-point arithmetic and rounding at every step, so it is slower than Bresenham's algorithm, and round-off error accumulates for long lines (the line may drift).

C5. Explain the scan-line polygon fill algorithm, with the help of a diagram/example.[5 Marks]

Answer:

Idea: for every horizontal scan line that crosses the polygon, find the points where the scan line intersects the polygon edges, sort the intersections by x, and fill the pixels between successive *pairs* of intersections (1st–2nd, 3rd–4th, ...). This uses the even-odd rule: between the 1st and 2nd intersection the scan line is inside the polygon, between the 2nd and 3rd it is outside, and so on.

Data structures

- **Edge Table (ET):** for every non-horizontal edge, stores y_{max}, the x value at y_{min}, and the inverse slope $1/m$. Edges are bucketed by y_{min}.

- **Active Edge List (AEL):** the edges crossed by the current scan line, kept sorted by x.

Algorithm

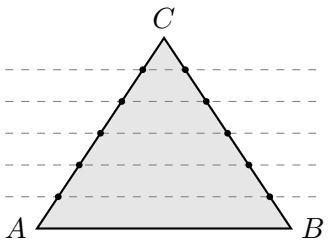
1. Build the ET (ignore horizontal edges) and start at the lowest scan line $y = y_{min}$ of the polygon.
2. Move edges from the ET whose $y_{min} = y$ into the AEL; sort the AEL by x.
3. Fill the pixels between pairs of x values in the AEL.
4. Increase y by 1. Remove edges with $y > y_{max}$. For the remaining edges update $x \leftarrow x + 1/m$.
5. Repeat from step 2 until the ET and AEL are empty.

Vertex handling: where two edges meet at a vertex, the vertex is counted once if the edges lie on opposite sides of the scan line (shorten one edge by one scan line) and twice (or zero times) at a local minimum/maximum; otherwise the parity of the crossings would be wrong.

Example

Triangle $A(2, 2)$, $B(10, 2)$, $C(6, 8)$. Edge AB is horizontal and ignored.

Edge	y_{min}	y_{max}	x at y_{min}	$1/m$
AC	2	8	2	$4/6 = 0.667$
BC	2	8	10	$-4/6 = -0.667$



Scan line y	x_{left}	x_{right}
2	2	10
3	2.67	9.33
4	3.33	8.67
5	4	8
6	4.67	7.33
7	5.33	6.67
8	6	6

At each scan line the pixels between x_{left} and x_{right} are filled (for $y = 3$: pixels $x = 3, \ldots, 9$). The x values are obtained incrementally by adding ± 0.667 per scan line, so no line-intersection formula is solved each time.

C6. Explain the inside–outside test using the Even–Odd Rule and the Winding Number Rule. How do the two rules differ for self-intersecting polygons? [5 Marks]

Answer:

An **inside–outside test** decides whether a given point P lies inside a polygon. It is needed for region filling, especially when the polygon is complex or self-intersecting. For simple polygons all rules agree, but for self-intersecting polygons the result depends on the rule used.

1. Even–Odd Rule (odd parity rule)

1. Draw a ray from P to infinity in any direction that does not pass through a vertex.
2. Count the number of times the ray crosses the polygon edges.
3. If the count is **odd**, P is **inside**; if it is **even** (including zero), P is **outside**.

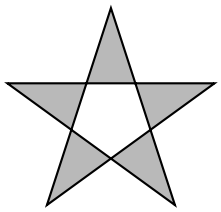
2. Winding Number Rule (non-zero winding rule)

1. Draw a ray from P to infinity and set the winding number $w = 0$.
2. Walk along the ray; for every polygon edge crossing the ray from *right to left* (as seen from the ray's direction, i.e. counter-clockwise around P) add $+1$, and for every edge crossing from *left to right* (clockwise) subtract 1.
3. If $w \neq 0$ then P is **inside**; if $w = 0$ then P is **outside**.

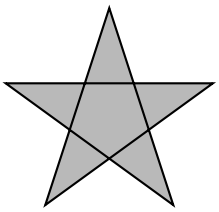
The winding number is the number of times the polygon boundary winds around P.

Difference for self-intersecting polygons

Take a five-pointed star drawn with a single continuous stroke (vertices joined in the order 1–3–5–2–4–1). The central pentagon is enclosed by the boundary *twice*.



Even–Odd: centre unfilled



Winding Number: centre filled

Region of star	**Even–Odd Rule**	**Winding Number Rule**
Five tips	1 crossing (odd) ⇒ inside	$w = \pm 1 \neq 0 \Rightarrow$ inside
Central pentagon	2 crossings (even) ⇒ **outside** (hole)	$w = \pm 2 \neq 0 \Rightarrow$ **inside** (filled)
Outside of star	0 crossings ⇒ outside	$w = 0 \Rightarrow$ outside

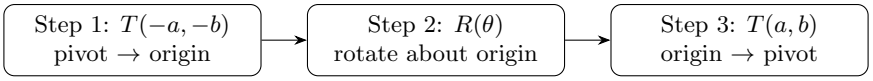
Conclusion: both rules fill the tips, but the Even–Odd Rule leaves the overlapping central region unfilled, while the Winding Number Rule fills it. The winding number rule can also distinguish the direction of the boundary (so two overlapping loops with opposite directions give $w = 0$ and a hole), while the even–odd rule is simpler and faster to compute.

Unit 3

C7. Derive the composite transformation matrix required to rotate a 2D object about an arbitrary point (a, b) by angle θ. [5 Marks]

Answer:

The standard rotation matrix $R(\theta)$ rotates about the *origin*. To rotate about (a, b) the pivot is first moved to the origin, the rotation is applied, and the pivot is moved back. With column vectors the matrices are applied right to left.



$$M = T(a,b) \cdot R(\theta) \cdot T(-a,-b) = \begin{bmatrix} 1 & 0 & a \\ 0 & 1 & b \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -a \\ 0 & 1 & -b \\ 0 & 0 & 1 \end{bmatrix}.$$

Step A: multiply the right two matrices.

$$R(\theta)\,T(-a,-b) = \begin{bmatrix} \cos\theta & -\sin\theta & -a\cos\theta + b\sin\theta \\ \sin\theta & \cos\theta & -a\sin\theta - b\cos\theta \\ 0 & 0 & 1 \end{bmatrix}.$$

Step B: pre-multiply by $T(a,b)$ (this adds a and b to the translation column).

$$\boxed{M = \begin{bmatrix} \cos\theta & -\sin\theta & a(1-\cos\theta) + b\sin\theta \\ \sin\theta & \cos\theta & b(1-\cos\theta) - a\sin\theta \\ 0 & 0 & 1 \end{bmatrix}}$$

Hence for a point (x, y):

$$x' = a + (x-a)\cos\theta - (y-b)\sin\theta, \qquad y' = b + (x-a)\sin\theta + (y-b)\cos\theta.$$

Check 1: the pivot maps to itself: $x' = a$, $y' = b$ for $(x, y) = (a, b)$.

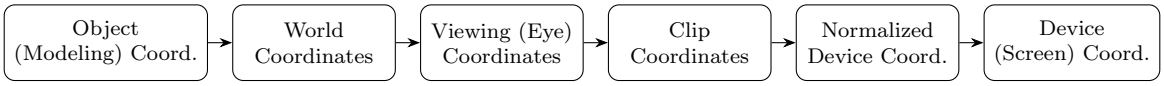
Check 2 (numerical): rotate $P(4,3)$ about $(2,1)$ by $90°$. With $\cos\theta = 0$, $\sin\theta = 1$: translation column $= (a+b,\ b-a) = (3,-1)$, so $x' = -y + 3 = 0$ and $y' = x - 1 = 3$. Result: $P'(0,3)$. (Direct: $x' = 2 + 0 - (2)(1) = 0$, $y' = 1 + (2)(1) + 0 = 3$. ✓)

Note: the order matters, because matrix multiplication is not commutative; $RT \neq TR$ in general. The same $T \cdot (\cdot) \cdot T^{-1}$ pattern is used for scaling about a fixed point and reflection about an arbitrary line.

C8. Explain the viewing pipeline, listing all coordinate systems involved from object coordinates to device coordinates. [5 Marks]

Answer:

The **viewing pipeline** is the sequence of transformations and operations that converts the description of an object, given in its own coordinate system, into the pixel positions shown on the display device.



1. **Object (Modeling/Local) coordinates:** every object is defined in its own convenient coordinate system (e.g. a square centred at its own origin).
2. **World coordinates:** the *modeling transformation* (scale, rotate, translate) places the objects in one common scene.
3. **Viewing (Eye/Camera) coordinates:** the *viewing transformation* positions and orients the scene relative to the camera, which is placed at the origin. In 2D this is the specification of the clipping *window* in world coordinates.
4. **Clip coordinates:** the *projection transformation* (orthographic or perspective) maps the view volume to a standard shape; *clipping* removes everything outside the window/view volume.
5. **Normalized device coordinates (NDC):** after the perspective divide by w, coordinates are mapped to a standard range such as $[0,1]$ or $[-1,1]$, independent of the actual display.
6. **Device (screen) coordinates:** the *window-to-viewport (viewport) transformation* maps NDC to pixel positions on the display:

$$x_v = x_{vmin} + (x_w - x_{wmin})\frac{x_{vmax} - x_{vmin}}{x_{wmax} - x_{wmin}}, \quad y_v = y_{vmin} + (y_w - y_{wmin})\frac{y_{vmax} - y_{vmin}}{y_{wmax} - y_{wmin}}.$$

Example: window $(0,0)$–$(100,100)$ and viewport $(0,0)$–$(800,600)$: the window point $(50,50)$ maps to $x_v = 0 + 50 \cdot 8 = 400$, $y_v = 0 + 50 \cdot 6 = 300$, i.e. the centre of the viewport.

2D version: World coordinates → window (viewing) transformation → clipping → normalization → viewport transformation → device coordinates.

C9. Explain the Cohen–Sutherland line-clipping algorithm, including its region-code scheme and the trivial accept/reject tests. [5 Marks]

Answer:

The **Cohen–Sutherland algorithm** clips a line against a rectangular window (x_{min}, y_{min})–(x_{max}, y_{max}). It first quickly accepts or rejects lines using 4-bit region codes, and computes intersections only for lines that need it.

Region-code scheme

The plane is divided into nine regions by extending the window edges. Every end point gets a 4-bit code **TBRL**:

Bit	Set to 1 if	Meaning
Bit 4 (T)	$y > y_{max}$	above the window
Bit 3 (B)	$y < y_{min}$	below the window
Bit 2 (R)	$x > x_{max}$	right of the window
Bit 1 (L)	$x < x_{min}$	left of the window

1001	1000	1010
0001	**0000**	0010
0101	0100	0110

Tests (codes c_1, c_2 of the two end points)

- **Trivial accept:** c_1 `OR` $c_2 = 0000$ (both codes are 0000) ⇒ the whole line is inside; draw it without clipping.
- **Trivial reject:** c_1 `AND` $c_2 \neq 0000$ ⇒ both end points lie on the outer side of the same window edge; discard the line.
- **Otherwise** the line is partly inside and must be clipped.

Algorithm

1. Compute the region codes c_1, c_2 of the end points.
2. Apply the accept/reject tests; stop if either applies.
3. Pick an end point with a non-zero code and find the intersection with the window edge it violates (check in the order T, B, R, L). With $m = \dfrac{y_2 - y_1}{x_2 - x_1}$:
   - for edge $y = y_{edge}$ (top/bottom): $x = x_1 + \dfrac{y_{edge} - y_1}{m}$;

- for edge $x = x_{edge}$ (left/right): $y = y_1 + m\,(x_{edge} - x_1)$.

4. Replace that end point by the intersection point, recompute its code, and repeat from step 2.

At most four iterations are needed (one for each window edge).

Example

Window $x_{min} = 2$, $x_{max} = 8$, $y_{min} = 2$, $y_{max} = 6$; line $P_1(1, 3)$–$P_2(9, 7)$.

- Codes: P_1: $x < 2 \Rightarrow 0001$; P_2: $x > 8,\ y > 6 \Rightarrow 1010$. OR $\neq 0$ and AND $= 0000$, so clipping is needed. $m = \frac{7-3}{9-1} = 0.5$.
- Clip P_1 against the left edge $x = 2$: $y = 3 + 0.5(2 - 1) = 3.5 \Rightarrow (2, 3.5)$, code 0000.
- Clip P_2 against the top edge $y = 6$: $x = 1 + \frac{6-3}{0.5} = 7 \Rightarrow (7, 6)$, code 0000.
- Both codes are 0000 $\Rightarrow$ accept: the visible line is $(2, 3.5)$–$(7, 6)$.

Section D — Long Answer Questions (7 Marks Each)

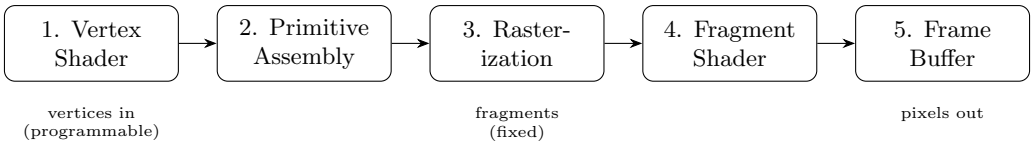
Unit 1

D1. Explain in detail the five stages of the GPU rendering pipeline. How does the concept of rasterization introduced in Unit 1 connect to the rasterization algorithms studied in Unit 2? [7 Marks]

Answer:

Introduction

The **GPU rendering pipeline** is the sequence of stages through which the geometry of a 3D scene (vertices, triangles, textures, lights) is turned into the 2D grid of coloured pixels displayed on the screen. A GPU contains thousands of small cores, so each stage processes many vertices or fragments in parallel, which makes real-time rendering possible. Some stages are *programmable* (shaders written by the programmer) and others are *fixed-function* hardware.



Stage 1: Vertex Shader (programmable)

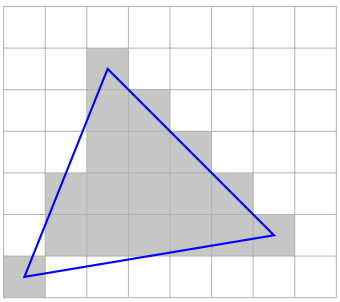
- **Input:** one vertex at a time with its attributes: position, normal, colour, texture coordinates.
- **Work:** the position is transformed through the *Model* (object → world), *View* (world → camera) and *Projection* (camera → clip space) matrices: $v_{clip} = P \cdot V \cdot M \cdot v$. It may also compute per-vertex lighting, animate vertices (skinning, waves) and pass data on to later stages.
- **Output:** vertices in clip-space coordinates (x, y, z, w).

Stage 2: Primitive Assembly (fixed-function)

- Vertices are connected into primitives (points, lines, triangles) according to the draw mode.
- **Clipping** removes or trims primitives outside the view volume; the **perspective divide** $(x/w, y/w, z/w)$ gives normalized device coordinates; the **viewport transform** gives screen positions; **back-face culling** discards triangles facing away from the camera.

Stage 3: Rasterization (fixed-function)

- Determines which screen pixels are covered by each triangle (by testing pixel centres against the triangle edges) and creates one *fragment* per covered pixel.
- Interpolates the vertex attributes (colour, depth z, texture coordinates) across the triangle using barycentric coordinates α, β, γ: $a = \alpha a_0 + \beta a_1 + \gamma a_2$.
- **Output:** a stream of fragments with interpolated values.



A triangle rasterized into fragments (shaded = pixel centre inside)

Stage 4: Fragment (Pixel) Shader (programmable)

- Executed once for every fragment. It samples textures, computes per-pixel lighting (e.g. Phong or physically based), shadows, normal mapping, fog and transparency, and produces the final colour (and optionally a new depth value) of the fragment.

Stage 5: Frame Buffer / Output Merger

- Per-fragment tests: scissor test, **depth test** (Z-buffer; only the nearest fragment is kept, which gives hidden surface removal) and stencil test.
- **Blending** combines the fragment colour with the colour already stored (for transparency).
- The final colour is written into the **frame buffer**; double buffering swaps front and back buffers to avoid flicker. The frame buffer content is sent to the display.

Connection between rasterization in Unit 1 and the algorithms of Unit 2

- In Unit 1, rasterization is a **pipeline stage**: it converts vector primitives (vertices of lines, triangles) into pixels. It tells *what* must be done.
- In Unit 2, the scan-conversion algorithms (DDA, Bresenham's line, circle and midpoint ellipse algorithms, polygon scan-line fill, flood/boundary fill) are the **methods** that decide *how* to select the pixels that best represent a line, curve or area on the discrete pixel grid.

Rasterization task in the pipeline	**Unit 2 algorithm used (conceptually)**
Drawing line primitives	DDA / Bresenham's line algorithm (incremental, integer decisions)
Filling the inside of a triangle/polygon	Scan-line polygon fill (edge intersections per scan line, fill between pairs; GPUs do this with edge functions in parallel)
Deciding inside/outside of a pixel	Inside–outside tests (even–odd, winding number)
Curved primitives (circle, ellipse)	Bresenham circle / midpoint ellipse algorithms
Removing jagged edges	Anti-aliasing (SSAA, MSAA) added at the rasterization/output stage

So the Unit 2 algorithms are the mathematical foundation of the rasterization stage; modern GPUs implement the same ideas in hardware and run them in parallel for millions of pixels.

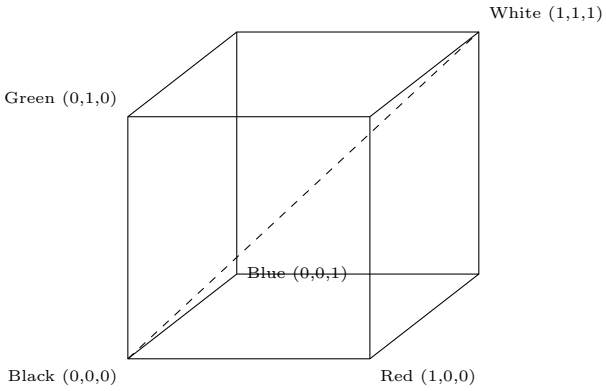
D2. Explain the following colour models in detail, with formulas/diagrams where applicable: (a) RGB (b) CMYK (c) HSV. Also discuss why accessibility should be considered when designing colour-based interfaces. [7 Marks]

Answer:

A **colour model** is a mathematical way of describing colours as numbers. Different models are suited to different devices and tasks.

(a) RGB colour model — additive

- The primaries are Red, Green and Blue *light*. The colour of a pixel is $C = R\,\mathbf{r} + G\,\mathbf{g} + B\,\mathbf{b}$ with each component in $[0, 1]$ (or 0–255 with 8 bits). A 24-bit image therefore has $256^3 = 16{,}777{,}216$ colours.
- The model is represented by a unit cube: corners $(0,0,0)$ = black, $(1,0,0)$ = red, $(0,1,0)$ = green, $(0,0,1)$ = blue, $(1,1,0)$ = yellow, $(0,1,1)$ = cyan, $(1,0,1)$ = magenta and $(1,1,1)$ = white. The diagonal from black to white contains the greys (t,t,t).
- **Use:** devices that emit light: monitors, TVs, phones, projectors, cameras, scanners and all screen graphics.



(The drawing is schematic: the cube's three axes carry R, G and B.)

(b) CMYK colour model — subtractive

- Cyan, Magenta and Yellow inks *absorb* red, green and blue light respectively from the white light reflected by paper; adding ink therefore makes the colour darker.
- Relation to RGB: $C = 1 - R,\ M = 1 - G,\ Y = 1 - B$. For example orange $(R, G, B) = (1, 0.5, 0)$ gives $(C, M, Y) = (0, 0.5, 1)$.
- In practice a black ink is added because C+M+Y does not give a true black (see B3). The conversion with black extraction is

$$K = 1 - \max(R, G, B), \qquad C = \frac{1 - R - K}{1 - K}, \quad M = \frac{1 - G - K}{1 - K}, \quad Y = \frac{1 - B - K}{1 - K}.$$

  For orange $(1, 0.5, 0)$: $K = 0$, so $(C, M, Y, K) = (0, 0.5, 1, 0)$.
- **Use:** colour printing (inkjet, laser, offset), publishing and packaging. The printable gamut is smaller than the RGB gamut, so very bright screen colours cannot be printed exactly.

(c) HSV colour model — perceptual

- **Hue** H (0°–360°): the colour type on the colour wheel: red 0°, yellow 60°, green 120°, cyan 180°, blue 240°, magenta 300°.

- **Saturation** S (0–1): purity of the colour; 0 is grey, 1 is fully vivid.
- **Value** V (0–1): brightness; 0 is black.
- The model is a cone/hexcone: hue is the angle around the axis, saturation the distance from the axis and value the height.
- **Conversion from RGB** (components in $[0, 1]$). Let $M = \max(R, G, B)$, $m = \min(R, G, B)$, $\Delta = M - m$:

$$V = M, \qquad S = \begin{cases} 0, & M = 0 \\ \Delta / M, & \text{otherwise} \end{cases} \qquad H = 60^\circ \times \begin{cases} \left(\dfrac{G - B}{\Delta} \right) \bmod 6, & M = R \\ \dfrac{B - R}{\Delta} + 2, & M = G \\ \dfrac{R - G}{\Delta} + 4, & M = B \end{cases}$$

  (H is undefined when $\Delta = 0$, i.e. for greys.) Example: orange $(1, 0.5, 0)$ has $M = 1$, $m = 0$, $\Delta = 1$, so $V = 1$, $S = 1$, $H = 60^\circ \times 0.5 = 30^\circ$.
- **Use:** colour pickers, photo editing (adjust brightness or saturation without changing the hue), and colour-based segmentation and tracking in computer vision.

	RGB	**CMYK**	**HSV**
Type	Additive	Subtractive	Perceptual (user-oriented)
Components	Red, Green, Blue	Cyan, Magenta, Yellow, Black	Hue, Saturation, Value
Medium	Emitted light (screens)	Reflected light (print)	Colour selection/editing

Accessibility in colour-based interface design

- **Colour vision deficiency:** roughly 8% of men and 0.5% of women have some form of colour blindness (most often red–green). Colours that look clearly different to a designer may look identical to them. Many other users have low vision, or use screens in bright sunlight or on low-quality displays.
- **Do not use colour as the only means of conveying information:** add text labels, icons, patterns or shapes. Example: an error is shown with a red colour *and* a warning icon *and* a message; chart lines use different markers or dash styles.
- **Adequate contrast:** the Web Content Accessibility Guidelines (WCAG) require a contrast ratio of at least 4.5 : 1 for normal text and 3 : 1 for large text (level AA); 7 : 1 for level AAA. Contrast depends mainly on lightness (the V component), not on hue.
- **Safe palettes:** avoid red–green and blue–purple pairs that differ only in hue; vary lightness; test with colour-blindness simulators; offer dark mode and high-contrast mode.
- **Benefits:** accessible design is often a legal requirement, widens the audience and improves usability for all users.

Unit 2

D3. Explain Bresenham's line-drawing algorithm in detail. Using the algorithm, compute the pixel positions for a line from $(2, 2)$ to $(8, 5)$, showing the decision parameter at each step. [7 Marks]

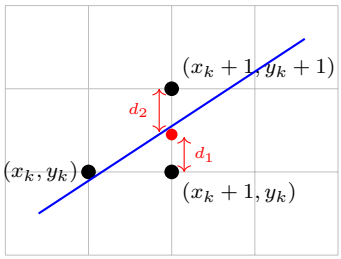
Answer:

Introduction

Bresenham's line algorithm is an incremental scan-conversion algorithm that draws a line using *only integer addition, subtraction and shifts*: no floating-point arithmetic, division or rounding. At each step it chooses between two candidate pixels by testing the sign of an integer **decision parameter** that measures which candidate is closer to the true line.

Derivation (for slope $0 < m < 1$, $x_1 < x_2$)

Let the current pixel be (x_k, y_k). The next column is x_k+1 and the two candidate pixels are $(x_k+1,\ y_k)$ and $(x_k+1,\ y_k+1)$.



With the line $y = mx + c$, at $x = x_k + 1$ the true value is $y = m(x_k + 1) + c$. The vertical distances to the two candidates are

$$d_1 = y - y_k = m(x_k + 1) + c - y_k, \qquad d_2 = (y_k + 1) - y = y_k + 1 - m(x_k + 1) - c.$$

$$d_1 - d_2 = 2m(x_k + 1) - 2y_k + 2c - 1.$$

If $d_1 < d_2$ the lower pixel y_k is closer, otherwise the upper pixel $y_k + 1$ is closer, i.e. we test the sign of $d_1 - d_2$. Multiplying by $dx > 0$ ($m = dy/dx$) removes the division and defines the integer decision parameter

$$p_k = dx\,(d_1 - d_2) = 2\,dy\,x_k - 2\,dx\,y_k + C, \qquad C = 2dy + dx(2c - 1) \text{ (a constant).}$$

- If $p_k < 0$ ($d_1 < d_2$): plot $(x_k + 1,\ y_k)$.
- If $p_k \geq 0$: plot $(x_k + 1,\ y_k + 1)$.

Recurrence: $p_{k+1} - p_k = 2dy\,(x_{k+1} - x_k) - 2dx\,(y_{k+1} - y_k) = 2dy - 2dx\,(y_{k+1} - y_k)$, hence

$$p_{k+1} = \begin{cases} p_k + 2\,dy, & p_k < 0 \quad (y \text{ unchanged}) \\ p_k + 2\,dy - 2\,dx, & p_k \geq 0 \quad (y \leftarrow y + 1) \end{cases}$$

Initial value: evaluating p at the first point (x_1, y_1), which lies on the line ($c = y_1 - mx_1$), gives

$$\boxed{p_0 = 2\,dy - dx}.$$

Algorithm (slope $0 < m < 1$)

```
1. Input (x1,y1), (x2,y2). dx = x2-x1, dy = y2-y1.
2. p = 2*dy - dx;  x = x1; y = y1.
3. Plot (x, y).
4. Repeat dx times:
       x = x + 1
       if p < 0:  p = p + 2*dy
       else:      y = y + 1;  p = p + 2*dy - 2*dx
       Plot (x, y)
```

Other slopes: for $m > 1$ the roles of x and y are exchanged (step in y, decision on x); for negative slopes y is decremented instead of incremented; lines with $x_1 > x_2$ are drawn by swapping the end points.

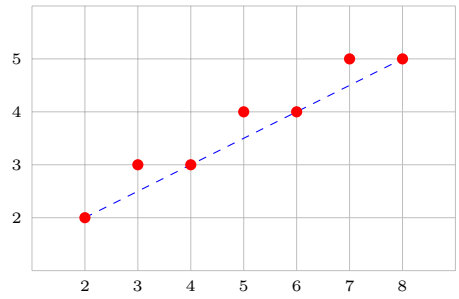
Numerical: line from $(2, 2)$ to $(8, 5)$

$dx = 8 - 2 = 6$, $dy = 5 - 2 = 3$, $m = 0.5 < 1$.

$$2\,dy = 6, \qquad 2\,dy - 2\,dx = 6 - 12 = -6, \qquad p_0 = 2\,dy - dx = 6 - 6 = 0.$$

k	p_k	Condition	Next pixel (x_{k+1}, y_{k+1})	p_{k+1}
			(2, 2) *(start)*	
0	0	$p_k \geq 0$	(3, 3)	$0 + (-6) = -6$
1	-6	$p_k < 0$	(4, 3)	$-6 + 6 = 0$
2	0	$p_k \geq 0$	(5, 4)	$0 - 6 = -6$
3	-6	$p_k < 0$	(6, 4)	$-6 + 6 = 0$
4	0	$p_k \geq 0$	(7, 5)	$0 - 6 = -6$
5	-6	$p_k < 0$	(8, 5)	$-6 + 6 = 0$

Pixels plotted: $(2, 2)$, $(3, 3)$, $(4, 3)$, $(5, 4)$, $(6, 4)$, $(7, 5)$, $(8, 5)$.



(When $p_k = 0$ the line passes exactly midway between two pixels; the convention $p_k \geq 0 \Rightarrow$ upper pixel is used.)

Advantages over DDA

	DDA	**Bresenham**
Arithmetic	Floating point (division, rounding)	Integer addition/subtraction only
Speed	Slower	Faster, easy to implement in hardware
Accuracy	Round-off error accumulates	No accumulated error; always picks the nearest pixel

D4. Explain Bresenham's circle-drawing algorithm in detail. Discuss the concept of 8-way symmetry and derive the initial decision parameter for a circle of radius 5. [7 Marks]

Answer:

Introduction

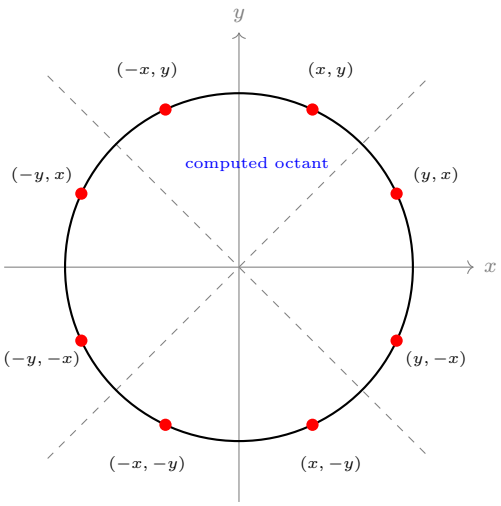
Drawing a circle by the equation $y = \pm\sqrt{r^2 - x^2}$ needs square roots, and drawing it by $x = r\cos\theta,\ y = r\sin\theta$ needs trigonometric functions: both are slow and leave gaps. **Bresenham's circle algorithm** generates the pixels with integer arithmetic only, using a decision parameter, and it computes only one octant of the circle.

8-way symmetry

A circle centred at the origin is symmetric about the x-axis, the y-axis and the lines $y = x$ and $y = -x$. Hence, if (x, y) is on the circle, then so are the other seven points

$$(y, x),\ (y, -x),\ (x, -y),\ (-x, -y),\ (-y, -x),\ (-y, x),\ (-x, y).$$

So only the octant from $x = 0$ to $x = y$ (from the top point $(0, r)$ clockwise to the 45° point) has to be computed; the rest are obtained by reflection. For a circle with centre (x_c, y_c), the centre is added to each plotted point: $(x_c \pm x,\ y_c \pm y)$ and $(x_c \pm y,\ y_c \pm x)$.



Derivation of the decision parameter

Work in the octant starting at $(0, r)$. In this octant x increases by 1 at every step, while y either stays the same or decreases by 1. If (x_i, y_i) is the current pixel, the two candidates are

$$T = (x_i + 1,\ y_i)\ \ \text{(keep } y\text{)}, \qquad S = (x_i + 1,\ y_i - 1)\ \ \text{(decrease } y\text{)}.$$

Define the circle function $D(x, y) = x^2 + y^2 - r^2$, which is > 0 outside, $= 0$ on, and < 0 inside the circle. The errors of the candidates are

$$D(T) = (x_i + 1)^2 + y_i^2 - r^2\ (\geq 0,\ \text{outside}), \qquad D(S) = (x_i + 1)^2 + (y_i - 1)^2 - r^2\ (< 0,\ \text{inside}).$$

The pixel with the smaller absolute error is closer to the circle. The decision parameter is the sum

$$d_i = D(T) + D(S) = |D(T)| - |D(S)| = 2(x_i + 1)^2 + y_i^2 + (y_i - 1)^2 - 2r^2.$$

- $d_i < 0$: $|D(T)| < |D(S)|$, so T is closer: choose $(x_i + 1, y_i)$.
- $d_i \geq 0$: S is closer: choose $(x_i + 1, y_i - 1)$.

Initial parameter at the start point $(x_0, y_0) = (0, r)$:

$$d_0 = 2(0 + 1)^2 + r^2 + (r - 1)^2 - 2r^2 = 2 + r^2 + r^2 - 2r + 1 - 2r^2 = \boxed{3 - 2r}.$$

For **radius** $r = 5$:

$$\boxed{d_0 = 3 - 2(5) = 3 - 10 = -7}.$$

Incremental update (from $d_{i+1} - d_i$):

- $d_i < 0$ (y unchanged): $d_{i+1} = d_i + 4x_i + 6$.
- $d_i \geq 0$ ($y \leftarrow y - 1$): $d_{i+1} = d_i + 4(x_i - y_i) + 10$.

(Proof for the first case: $d_{i+1} - d_i = 2[(x_i + 2)^2 - (x_i + 1)^2] = 2(2x_i + 3) = 4x_i + 6$. For the second case the extra terms $[(y_i - 1)^2 - y_i^2] + [(y_i - 2)^2 - (y_i - 1)^2] = -4y_i + 4$ are added, giving $4x_i + 6 - 4y_i + 4 = 4(x_i - y_i) + 10$.)

Algorithm

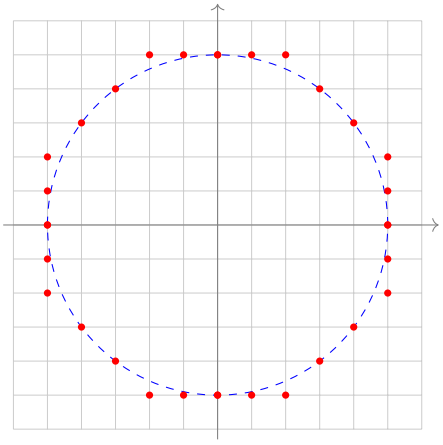
```
1. Input radius r and centre (xc, yc).
2. x = 0; y = r; d = 3 - 2*r.
3. While x <= y:
       Plot the 8 symmetric points (xc +/- x, yc +/- y), (xc +/- y, yc +/- x)
       if d < 0:  d = d + 4*x + 6
       else:      d = d + 4*(x - y) + 10;  y = y - 1
       x = x + 1
```

Example: $r = 5$, centre at the origin

i	(x_i, y_i)	d_i	Rule	Next (x_{i+1}, y_{i+1}) and d_{i+1}
0	(0, 5)	-7	$d < 0$: $d + 4x + 6$	(1, 5), $d = -7 + 0 + 6 = -1$
1	(1, 5)	-1	$d < 0$: $d + 4x + 6$	(2, 5), $d = -1 + 4 + 6 = 9$
2	(2, 5)	9	$d \geq 0$: $d + 4(x - y) + 10$	(3, 4), $d = 9 - 12 + 10 = 7$
3	(3, 4)	7	$d \geq 0$	next would be $(4, 3)$; $x > y$, so **stop**

Points in the first octant: $(0, 5), (1, 5), (2, 5), (3, 4)$. Applying the 8-way symmetry gives all points of the circle:

- from $(0, 5)$: $(0, 5), (0, -5), (5, 0), (-5, 0)$;
- from $(1, 5)$: $(\pm 1, \pm 5)$ and $(\pm 5, \pm 1)$;
- from $(2, 5)$: $(\pm 2, \pm 5)$ and $(\pm 5, \pm 2)$;
- from $(3, 4)$: $(\pm 3, \pm 4)$ and $(\pm 4, \pm 3)$.



Advantages

Integer-only arithmetic, no square roots or trigonometry, only one octant computed (about 1/8 of the work), and a symmetric, gap-free circle.

D5. Explain the midpoint ellipse algorithm, including why it requires two separate regions. Compare it with the circle-drawing algorithm studied earlier. [7 Marks]

Answer:

Introduction

The **midpoint ellipse algorithm** draws an ellipse centred at the origin with semi-axes r_x (along x) and r_y (along y) using the sign of the ellipse function at the *midpoint* between two candidate pixels. The ellipse equation and the function used are

$$\frac{x^2}{r_x^2} + \frac{y^2}{r_y^2} = 1 \quad \Longrightarrow \quad f(x,y) = r_y^2 x^2 + r_x^2 y^2 - r_x^2 r_y^2.$$

$f < 0$ for a point inside, $f = 0$ on, and $f > 0$ outside the ellipse. For a centre (x_c, y_c) the plotted points are shifted by (x_c, y_c).

Symmetry

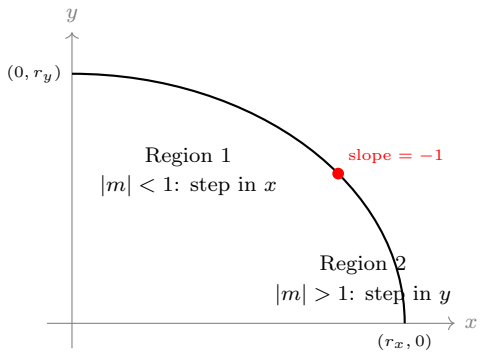
An ellipse has only **4-way symmetry** (about the two axes): if (x, y) is on the ellipse so are $(-x, y)$, $(x, -y)$ and $(-x, -y)$. The lines $y = \pm x$ are *not* axes of symmetry unless $r_x = r_y$. Hence one full quadrant must be computed.

Why two regions are required

In the first quadrant, starting at $(0, r_y)$ and moving clockwise to $(r_x, 0)$, the slope of the boundary is

$$\frac{dy}{dx} = -\frac{r_y^2\, x}{r_x^2\, y},$$

which changes continuously from 0 at $(0, r_y)$ to $-\infty$ at $(r_x, 0)$. The point where the slope is -1 separates two different behaviours.



- **Region 1** ($|m| < 1$, i.e. $2r_y^2 x < 2r_x^2 y$): the curve is flat, so x changes faster than y. We take *unit steps in* x and decide whether y stays or decreases by 1.
- **Region 2** ($|m| > 1$, i.e. $2r_y^2 x \geq 2r_x^2 y$): the curve is steep, so y changes faster than x. We take *unit steps in* y (downwards) and decide whether x stays or increases by 1.

If x were stepped in Region 2, consecutive pixels would be more than one pixel apart vertically and the ellipse would show gaps. A circle, in contrast, has the same radius in both directions, so within one octant the slope is always $|m| \leq 1$ and one region is enough.

Region 1 (start $(0, r_y)$)

Decision parameter at the midpoint $(x_k + 1,\ y_k - \frac{1}{2})$:

$$p1_k = f\left(x_k + 1,\ y_k - \tfrac{1}{2}\right), \qquad p1_0 = r_y^2 - r_x^2 r_y + \tfrac{1}{4} r_x^2.$$

- $p1_k < 0$: midpoint inside, choose $(x_k + 1,\ y_k)$; $p1_{k+1} = p1_k + 2r_y^2 x_{k+1} + r_y^2$.

- $p1_k \geq 0$: choose $(x_k + 1,\ y_k - 1)$; $p1_{k+1} = p1_k + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_y^2$.
- Stop Region 1 when $2r_y^2 x \geq 2r_x^2 y$.

Region 2 (start at the last point (x_0, y_0) of Region 1)

Decision parameter at the midpoint $(x_k + \frac{1}{2},\ y_k - 1)$:

$$p2_k = f\big(x_k + \tfrac{1}{2},\ y_k - 1\big)\,, \qquad p2_0 = r_y^2\big(x_0 + \tfrac{1}{2}\big)^2 + r_x^2(y_0 - 1)^2 - r_x^2 r_y^2.$$

- $p2_k > 0$: midpoint outside, choose $(x_k,\ y_k - 1)$; $p2_{k+1} = p2_k - 2r_x^2 y_{k+1} + r_x^2$.
- $p2_k \leq 0$: choose $(x_k + 1,\ y_k - 1)$; $p2_{k+1} = p2_k + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_x^2$.
- Stop when $y = 0$. At every step plot the four symmetric points $(\pm x, \pm y)$.

Example: $r_x = 8$, $r_y = 6$ (so $r_x^2 = 64$, $r_y^2 = 36$, $2r_y^2 = 72$, $2r_x^2 = 128$)

Region 1: $p1_0 = 36 - 64(6) + \frac{1}{4}(64) = 36 - 384 + 16 = -332$, start $(0, 6)$.

k	$p1_k$	Next pixel	$2r_y^2 x_{k+1}$	$2r_x^2 y_{k+1}$
0	-332	(1, 6)	72	768
1	-224	(2, 6)	144	768
2	-44	(3, 6)	216	768
3	208	(4, 5)	288	640
4	-108	(5, 5)	360	640
5	288	(6, 4)	432	512
6	244	(7, 3)	504	384

At $(7, 3)$: $504 \geq 384$, so Region 1 ends. Region 2: $p2_0 = 36(7.5)^2 + 64(2)^2 - 64 \cdot 36 = 2025 + 256 - 2304 = -23$, start $(7, 3)$.

k	$p2_k$	Next pixel
0	-23	(8, 2)
1	361	(8, 1)
2	297	(8, 0)

Points of the first quadrant: $(0,6), (1,6), (2,6), (3,6), (4,5), (5,5), (6,4), (7,3), (8,2), (8,1), (8,0)$; the other three quadrants follow by sign changes.

Comparison with the circle algorithm

Aspect	Circle (Bresenham/midpoint)	Midpoint ellipse
Parameters	One radius r	Two semi-axes r_x, r_y
Symmetry	8-way (one octant computed)	4-way (one quadrant computed)
Regions	One region (slope always $\lvert m\rvert \le 1$ in the octant)	Two regions (slope changes across -1)
Decision parameters	One (d or p)	Two ($p1$ in Region 1, $p2$ in Region 2)
Stepping axis	Always x	x in Region 1, y in Region 2
Stop condition	$x \ge y$	Switch when $2r_y^2 x \ge 2r_x^2 y$; stop at $y = 0$
Computation	Simpler, fewer terms	More terms and multiplications
Special case	–	For $r_x = r_y = r$ it becomes a circle

Unit 3

D6. Explain the following 2D transformations with their matrix representations using homogeneous coordinates: translation, rotation, scaling, reflection and shearing. [7 Marks]

Answer:

Introduction

A transformation changes the position, orientation, size or shape of an object by transforming its vertices. In **homogeneous coordinates** a point (x, y) is written $(x, y, 1)^T$, and every 2D transformation is a 3×3 matrix M with $P' = M \cdot P$. Using one common matrix form lets a sequence of transformations be concatenated into a single *composite* matrix by multiplication, applied to a point from right to left.

1. Translation

Moves every point by (t_x, t_y): $x' = x + t_x$, $y' = y + t_y$.

$$T(t_x, t_y) = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}, \qquad T^{-1} = T(-t_x, -t_y).$$

2. Rotation (about the origin, by angle θ, counter-clockwise positive)

Let $P = (r\cos\varphi,\ r\sin\varphi)$. After rotation by θ: $x' = r\cos(\varphi + \theta) = x\cos\theta - y\sin\theta$ and $y' = r\sin(\varphi + \theta) = x\sin\theta + y\cos\theta$.

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}, \qquad R^{-1} = R(-\theta) = R^T.$$

Rotation about a pivot (a, b): $T(a, b)\, R(\theta)\, T(-a, -b)$.

3. Scaling (about the origin, factors s_x, s_y)

$x' = s_x\, x$, $y' = s_y\, y$.

$$S(s_x, s_y) = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}, \qquad S^{-1} = S(1/s_x,\ 1/s_y).$$

$s_x = s_y$: uniform scaling (shape preserved); $s_x \neq s_y$: differential scaling (shape distorted); $s > 1$ enlarges, $0 < s < 1$ shrinks. Scaling about a fixed point (x_f, y_f): $T(x_f, y_f)\, S(s_x, s_y)\, T(-x_f, -y_f)$.

4. Reflection (mirror image)

A reflection produces the mirror image of the object about an axis or a line.

Reflection about	**Effect**	**Matrix**
x-axis	$(x, y) \to (x, -y)$	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$
y-axis	$(x, y) \to (-x, y)$	$\begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$
Origin	$(x, y) \to (-x, -y)$	$\begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$
Line $y = x$	$(x, y) \to (y, x)$	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$
Line $y = -x$	$(x, y) \to (-y, -x)$	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$

Reflection is a scaling with factor -1; reflection about the origin equals a rotation by 180°.

5. Shearing

A shear slants the shape: the x-shear shifts x in proportion to y, the y-shear shifts y in proportion to x.

$$Sh_x = \begin{bmatrix} 1 & sh_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \; (x' = x + sh_x\, y,\; y' = y), \qquad Sh_y = \begin{bmatrix} 1 & 0 & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \; (x' = x,\; y' = y + sh_y\, x).$$

Example: triangle $A(1,1)$, $B(3,1)$, $C(2,3)$

Transformation	A'	B'	C'
Translation $T(2,3)$	(3, 4)	(5, 4)	(4, 6)
Rotation $R(90°)$: $(x, y) \to (-y, x)$	$(-1, 1)$	$(-1, 3)$	$(-3, 2)$
Scaling $S(2,2)$	(2, 2)	(6, 2)	(4, 6)
Reflection about x-axis	$(1, -1)$	$(3, -1)$	$(2, -3)$
Shear Sh_x with $sh_x = 1$: $x' = x + y$	(2, 1)	(4, 1)	(5, 3)

Properties

- Translation, rotation and reflection are *rigid-body* transformations (they preserve lengths and angles); scaling and shearing do not.

- All five are *affine* transformations: they preserve straight lines and parallelism.
- Composite transformations are order-dependent because matrix multiplication is not commutative: "rotate, then translate" $\neq$ "translate, then rotate".

D7. Explain the Sutherland–Hodgman polygon-clipping algorithm in detail with a diagram, and state its limitations. [7 Marks]

Answer:

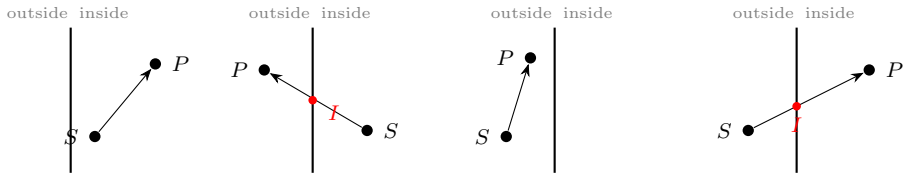
Introduction

Clipping a polygon is different from clipping a line: the result must again be a closed polygon (possibly with new edges along the window boundary). The **Sutherland–Hodgman algorithm** clips a polygon against a *convex* clip window by clipping it successively against **one boundary at a time** (left, right, bottom, top). The output vertex list of one boundary is the input list of the next, like a pipeline of four clippers.

Processing of one boundary

The vertices are processed in order, taking each pair of consecutive vertices as an edge from S (previous vertex) to P (current vertex); for the first vertex, S is the last vertex of the list. Each pair is classified by whether S and P are inside or outside the boundary:

Case	S	P	Output
1	inside	inside	P
2	inside	outside	intersection point I (edge leaves the window)
3	outside	outside	nothing
4	outside	inside	I and then P (edge enters the window)



Intersection point and inside test

For a vertical boundary $x = x_b$: $I_x = x_b$, $I_y = y_S + \dfrac{y_P - y_S}{x_P - x_S}(x_b - x_S)$. For a horizontal boundary $y = y_b$: $I_y = y_b$, $I_x = x_S + \dfrac{x_P - x_S}{y_P - y_S}(y_b - y_S)$. A point is *inside* the left boundary if $x \geq x_{min}$, right if $x \leq x_{max}$, bottom if $y \geq y_{min}$, top if $y \leq y_{max}$.

Algorithm

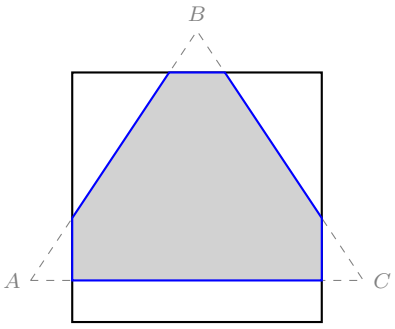
```
inputList = polygon vertices
for each clip boundary (left, right, bottom, top):
    outputList = empty
    S = last vertex of inputList
    for each vertex P in inputList:
        if P is inside:
            if S is outside: add intersection(S,P,boundary)
            add P
```

```
        else if S is inside:
            add intersection(S,P,boundary)
        S = P
    inputList = outputList
result = inputList
```

Worked example

Window: $x_{min} = 2$, $x_{max} = 8$, $y_{min} = 2$, $y_{max} = 8$. Triangle $A(1,3)$, $B(5,9)$, $C(9,3)$. Edge slopes: AB: $+1.5$, BC: -1.5.

Boundary	**Vertex list after clipping**
Left $x = 2$	Edge CA (in→out): $I = (2,3)$. Edge AB (out→in): $I = (2,4.5)$, then B. Edge BC (in→in): C. List: $(2,3), (2,4.5), (5,9), (9,3)$
Right $x = 8$	Edge $(9,3) \to (2,3)$ (out→in): $(8,3)$, then $(2,3)$. Edges to $(2,4.5)$ and $(5,9)$ (in→in): $(2,4.5)$, $(5,9)$. Edge $(5,9) \to (9,3)$ (in→out): $(8,4.5)$. List: $(8,3), (2,3), (2,4.5), (5,9), (8,4.5)$
Bottom $y = 2$	All vertices have $y \geq 2$ (inside): list unchanged
Top $y = 8$	Edge $(2,4.5) \to (5,9)$ (in→out): $(4.33,8)$. Edge $(5,9) \to (8,4.5)$ (out→in): $(5.67,8)$, then $(8,4.5)$. Other edges in→in. **Final:** $(8,3), (2,3), (2,4.5), (4.33,8), (5.67,8), (8,4.5)$



Dashed: original triangle; shaded: clipped polygon

Limitations

1. The clip window must be **convex** (the basic form uses a rectangle); a concave clip window gives wrong results.
2. For a **concave subject polygon** that should be split into two or more separate pieces after clipping, the algorithm produces a single polygon joined by extra *degenerate (overlapping) edges* along the clip boundary.
3. Because every boundary produces an intermediate vertex list, it needs extra memory and repeated intersection calculations (a pipelined or parallel hardware implementation helps).
4. It works for polygons only (closed vertex lists), not for open polylines or curves, and only for the specified inside/outside tests of simple boundaries.
5. For concave polygons or concave windows, the **Weiler–Atherton** algorithm is used instead.

— End of Solutions —